

# 1

# The Category Error



---

## OPENING CASE

### Two subscriptions, one correction

In the spring of 2025, a software team at a mid-size company was doing what thousands of teams were doing: paying twenty dollars per developer per month for an AI coding assistant called Cursor, and treating that line on the budget the way it treated every other software line. The tool was useful. The price was small. The arrangement was, on its face, a subscription like any other, filed next to the ticketing system and the design software and the password manager. Nobody in finance thought about it again after signature, because nothing about a twenty-dollar seat asks to be thought about.

Then, in June 2025, the arrangement changed underneath them. Cursor revised its pricing so that the flat monthly allotment became metered usage billed at the underlying model rates. For light users the change was invisible. For the team's heaviest users, the people getting the most value, the monthly allowance was consumed in a matter of prompts, after which usage continued to bill against real rates. Charges arrived that no one had planned for. The company's chief executive published an apology and an explanation. The explanation was not an excuse. It was an account of arithmetic: the upstream cost of serving heavy users under a flat price had become untenable, and the flat price had to go.

The same pattern, in the same window, played out at a far larger scale and with no apology at all. GitHub Copilot, the most widely adopted AI coding assistant in the world, spent the year from June 2025 to June 2026 dismantling its own flat model in two acts. First it capped request types that had effectively been unlimited. Then, across a paid base reported at 4.7 million subscribers, it retired flat premium requests entirely and moved billing to token-denominated credits. One vendor corrected its pricing with a public apology. The other corrected its pricing in two quiet steps across millions of accounts. The correction was the same.

Set the two episodes side by side and the shared feature is not the apology, the timing, or the size of the vendor. The shared feature is what broke. In both cases a product had been sold, and bought, as software: a seat, a flat price, a cost fixed at signature. In both cases the thing underneath the seat turned out to obey a different law. It consumed a resource on every use, the resource had a real and variable cost, and heavy use ran that cost past the flat price. When the gap grew large enough, the price changed. It changed on the provider's schedule, not the buyer's, and it changed whether or not the buyer had ever looked at a meter.

The buyers in these stories did not make an error of vendor selection. Cursor and Copilot were, and are, excellent tools. The buyers made an error of category. They filed a metered resource under the mental model they use for licensed software, and the mental model held right up until the moment it did not. This chapter is about that error: what it is, why it was reasonable,

and what the organization actually purchased when it thought it was buying software.

## 1.1 The purchase that is not one

The buyer brought a model to the transaction, and it was a good model, refined over three decades of enterprise software procurement. In that model, software is a license. The organization buys the right for a defined number of people to use a program. The marginal cost of an additional use, once the seat is paid for, is approximately zero: a licensed user who runs the program a thousand times a day costs the vendor essentially the same as one who runs it once a week. Cost is therefore fixed at signature. The negotiation that matters happens once, at purchase, over the number of seats and the price per seat. After that, management of the software is management of access: who has a login, how many seats are active, when the contract renews.

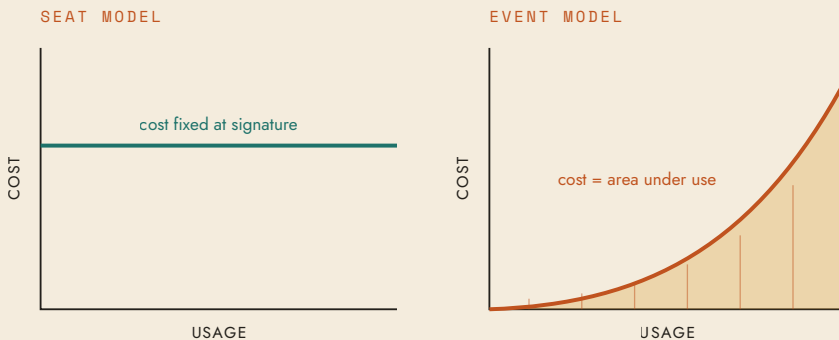
This model is not wrong about licensed software. It is wrong about the thing now being sold under the same packaging. What the organization operates when it deploys an AI assistant is not a program that people are licensed to run. It is work that consumes a metered resource on every task. Each use sends a request to a model, the model performs computation to answer it, and that computation has a cost that scales with the size of the request and the size of the answer. The marginal cost of an additional use is not approximately zero. It is the central fact.

The consequence is a mismatch between what the organization manages and what the organization operates. Access management asks how many people may use the tool. Resource management asks how much of the resource the work consumes, at what cost, for what return. An organization that manages access while operating a metered resource is managing the wrong quantity. It counts seats while the bill is written in tokens. The mental model determines what gets managed, and the wrong model manages the wrong thing.

## 1.2 The consumption event

The discipline that follows in this book is built on a single atomic unit, and it is worth defining precisely before anything is built on top of it. A **consumption event** is a request that consumes computational resource, metered in tokens or their equivalents, at a per-event cost greater than zero. Every deployed use of AI, at bottom, is a stream of these events. They are the thing the meter counts, the thing the invoice sums, and the thing every later chapter learns to source, record, attribute, budget, allocate, and hold accountable.

The anatomy of one event is simple and worth holding in view. Something goes in: a prompt, a document, a conversation history, retrieved reference material. The model performs computation. Something comes back: a completion, a suggestion, an answer, a tool call. A meter records what was consumed, and a record of that consumption is held somewhere. In the ordinary case, it is held by the provider, and the buyer sees only its monthly summation on an invoice.



**FIGURE 1.1** Two purchase models. Under the seat model, cost is flat against rising usage. Under the event model, cost is the accumulating sum of consumption events. The same usage produces a flat cost on the left and a rising cost on the right.

The seat model and the event model are not two views of the same economics. They are two different economics, and Figure 1.1 draws the difference the invoice hides. Under the seat model, cost is a horizontal line: usage

risers and cost does not follow. Under the event model, cost is the area under the usage curve: every event adds to the total, and heavy use is expensive by construction. A buyer operating on the left panel while the right panel governs the bill will be surprised, and the surprise will arrive as a number, on a schedule the buyer did not set.

### 1.3 The flat-rate objection, answered

There is a strong objection to everything said so far, and it deserves to be stated at full strength rather than in a weakened form built to be knocked down. The objection is this: we pay thirty dollars per seat per month, the price is flat, we are never billed by the token, and for us this simply is software.

The objection is not confused, and the answer is not that the buyer is billed by the token after all. The answer is that a flat rate relocates the meter; it does not abolish it. When a provider offers a flat price for a metered resource, the provider does not stop metering. The provider meters exactly what the buyer has declined to see, and prices the flat rate against an assumption about how much the average buyer will consume. That assumption is a bet. It fails when usage is skewed, when a minority of heavy users consume a majority of the resource, and the instability resolves in one direction: the provider corrects the price, on the provider's schedule, because the provider holds the meter and the buyer does not. This is not a claim about badly run vendors. It is a structural result, and the registry states it as a theorem.

THM-009

#### **AI Use Is Resource Consumption, Not Merely Software Access**

*If an AI activity requires compute and executing that activity consumes resources, production scaling expands the usage surface of AI deployment, buyer total cost extends beyond access price, and measurement enables visibility into resource use or cost-bearing activity, then AI use in*

*a deployed business context is resource-consuming operating activity, not merely software access, within the relevant deployment and cost boundary.*

The theorem does not say that a flat rate is impossible or that every subscription will be repriced tomorrow. It says something narrower and more durable: that in a deployed business context, the thing being operated is a resource-consuming activity rather than software access. The flat rate is a way of packaging that activity for sale. It is not evidence that the activity is something else. The buyer who reads the flat rate as proof of software access has mistaken the packaging for the contents.

## Key terms

**Consumption event.** A request that consumes computational resource, metered in tokens or their equivalents, at a per-event cost greater than zero. The atomic unit of the discipline.

**Software access model.** The economic model, inherited from licensed software, in which cost is fixed at signature and the marginal cost of additional use is approximately zero; management is management of access.

**Access price.** The stated price of access to an AI capability, per seat or per subscription, distinct from the total cost of consuming the resource behind it.

**Meter relocation.** The principle that a flat rate does not abolish the meter but moves it to the provider's side, where the provider prices the flat rate against assumed consumption and corrects it on its own schedule when usage is skewed.